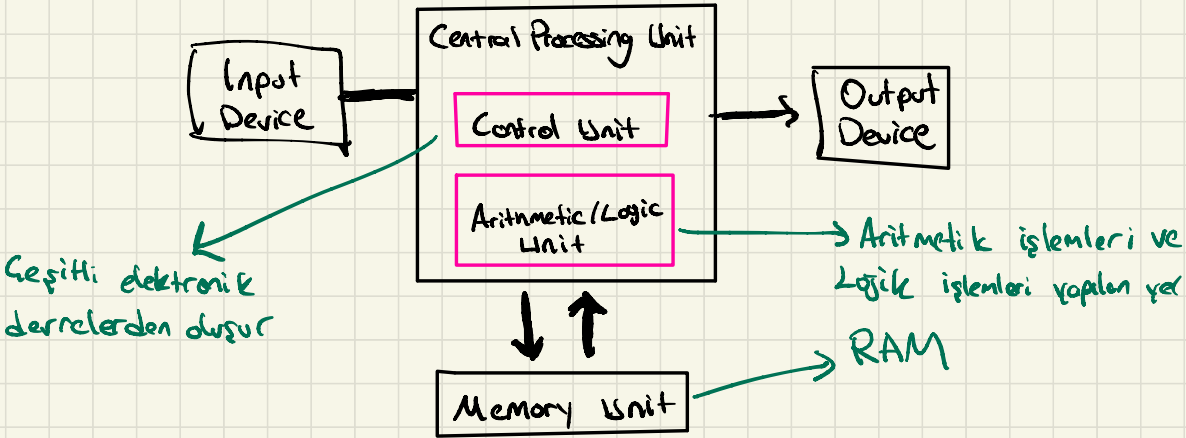


ALT Seviye Programlama

Von Neuman - 80x86



• Birimlerin bağlantıları yollar ile (bus) ile sağlanır.

- > Address Bus (Adress Yolu)
- > Data Bus (Veri Yolu)
- > Control Bus (Kontrol Yolu)

• Yolların tipleri

- > Paralel → Hızlı, kalın kablo
- > Seri → Yavaş, ince kablo

- FSB (Front Side Bus)

- İşlemci ile anakart arasındaki bağlantı

- Bellek Bağlantı Yolları

- SDRAM
- DDR-SDRAM
- DDR2-SDRAM
- ...

- Grafik Birim Bağlantı Yolları

- AGP
- PCIe

- İkincil Bellek Birim Bağlantı Yolları

- ATA
- SATA

- Diğer Cihaz Birim Bağlantı Yolları

- IEEE 1394
- APPLE → Firewire

- USB

80x86 Ailesi İşlemciler

- CISC (Complex Instruction Set Computers)
- İşlem hacmi → 8, 16, 32, 64
- Intel, Amd, Cytrix, IBM, TI vb.
- Backward Code Compatibility (Geriye Yönelik Kod Uyumu)
- Matematik İşlemciler → 8086 için 8087 vb.

80x86 İşlemcisinin İç Yapısı



Ax = Accumulator Register
Cx = Count Register
Bx = Base Register
Dx = Data Register

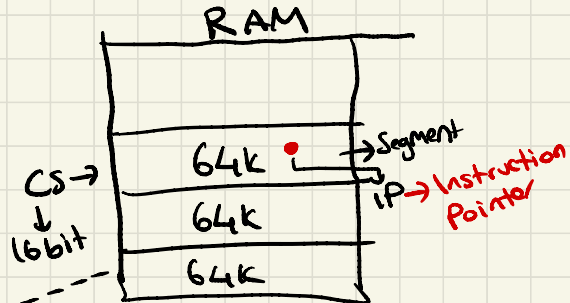
- Yalnızca Stone iç içe for dengisi kullanılır.

Bellekler, Yazmaçlar ve Fiziksel Adres Hesapları

$$2^{16} = 65536 \rightarrow 64 \text{ K} \rightarrow \text{Segment}$$
$$2^{20} = 1 \text{ MB} \rightarrow 2^{20} \rightarrow 2^{30} \rightarrow 1 \text{ G}$$

Handwritten calculations show the relationship between memory size and segment size:

$$2^{16} \times 2^{10} = 2^{26} \text{ K}$$
$$2^{20} \times 2^{10} = 2^{30} \text{ M}$$



Fiziksel Adres: $CS \times 16 + IP$

CS = 2C58H → H = Hexadecimal
IP = 0013H

CS = 2C50H 2C50H
IP = 0093H + 0093H

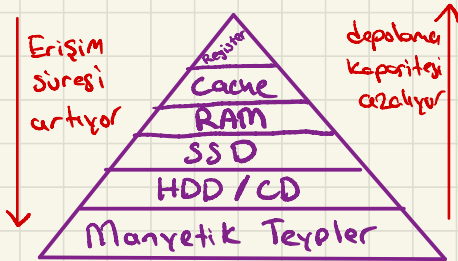
$$(2C58)_{16} \times (16)_{10} = (2C580)_{16} + 00013H$$

Fiziksel Adresimiz

2C593H

Fiziksel adres aynı!

2C593H



1. Hafta Bitti

2. Hafta Başlangıcı

Segmentler / Kesimler

- Kod Kesimi (Code Segment)
> CS:IP
- Yığın Kesimi (Stack Segment)
> SS:[SP-BP]
> LIFO: Last in first Out
> Push-Pop
- Veri Kesimi (Data Segment)
> DS:[SI, DI, BX]
- Ek Kesim (Extra Segment)
> ES:[SI, DI, BX]

CS
IP

SS
SP
BP

ES:DS
SI
DI
BX

Extra Segment, Data Segment yetersiz kaldığında kullanılabilir.

Bayraklar (Flags-PSW)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	OF	DF	IF	TP	SF	ZF	-	AF	-	PF	-	CF
Bayrak				Görevi (Set -> 1, Clear, Reset -> 0)											
CF: Carry Flag				Elde ödünç durumlarında CF set olur.											
PF: Parity Flag				Even parity -> Set, Odd Parity -> Clear (Even: Çift)											
AF: Auxilary CF				8 - bit işlemde 4'den 5'e; 16 - bit işlemde 8'den 9'a elde/ödünç aktarımı											
ZF: Zero Flag				İşlem sonucu 0 ise Set, 1 ise Clear											
SF: Sign Flag				İşlem sonucu negatif ise Set, pozitif ise Clear											
TP: Trap Flag				Adım bayrağı. Set ise her komuttan sonra kesme üretilerek prog. izleniyor											
IF: Interrupt Flag				Maskelenebilir (Maskable) kesmelerin kontrolü için kullanılır. 1 ise izin verilir.											
DF: Direction Flag				Dizgi (String) işlemlerinde işlemin yönünü belirlemek için kullanılır.											
OF: Overflow Flag				Aritmetik taşma durumunda Set, aksi durumda Clear.											

● Belleğe Erişim ●

- İşlemcinin adresleyebileceği bellek adres bacağı ile değişir.
> 20 bacak varsa; $2^{20} = 1\text{MB}$
- Fiziksel Adres Hesabı

Gerçek Kip

- 16-bitlik kesim yazmaçları sayesinde 64K'lık bloklara erişilebilir. Toplam alan 1MB'dır.
- Programlar birbirlerinin bellek alanına erişebilir. Multitasking (Çoklu görev) yoktur.
- Yazılım ile BIOS kodlarına veya donanıma doğrudan erişmek mümkündür.

Korunmalı Kip

- Bellek koruması vardır. Programlar birbirlerinin alanına giremez. Çoklu görev desteklenir.
- Donanım olarak, programın çalışma sıralarını değiştirme hakkı vardır (Preemptive Multitasking)
- Korunmalı kipte yazmaçlar 32-bitlik tir.
- Günümüz işletim sistemleri açılışı sırasında işlemciyi korunmalı kipe geçirerek çalışır.

İşleminin Komutları Adım Adım Çalıştırması

1. Komutu belirleyen byte'ın bellek üzerindeki kod alanına alınması (Instruction fetch)
2. IP yazmacının bir sonraki byte'ı gösterecek şekilde değiştirilmesi
3. Alınan komutun ne komutu olduğunun ve ne tür parametrelerle çalışacağını belirlenmesi (Instruction Decode)
4. Gerekli olması durumunda kullanılacak parametrelerin bellek alanından alınması (Operand Fetch)
5. IP yazmacının bir sonraki komutu gösterecek şekilde ayarlanması
6. İşlemin gerçekleştirilmesi (Execute)
7. Elde edilen sonucun gerekli olan yere yerleştirilmesi (Store)

Mnemonic (Assembly Komutları)

{ Etiket: } Mnemonic { { operand işlenen1, işlenen2 } ; açıklamalar

↓

Mov → Ax, 2
→ Ax, Bx
ADD → Cx, 7
SUB → Bp
INC
NOP → No operation

Veri Aktarım Komutları

MOV

LEA

LDS

LES

XCHG

MOV:

MOV OP₁, OP₂

OP₁ ← OP₂

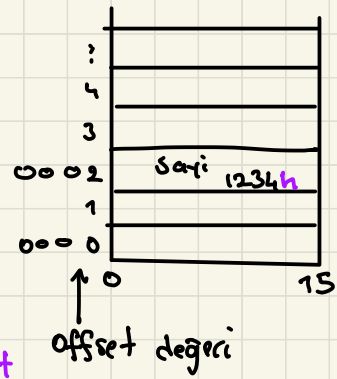
Operand 2'deki değeri operand 1'e atar.

LEA:

LEA, RW, DADDR

offset değerini Register'in
içine kopyalıyor

MOV da ise offsetin değerini
registere kopyalıyor.



LEA RW, DADDR

8-bit
8D

8-bit
aasssbbb

[DISP][DISP] 2+EA

object code

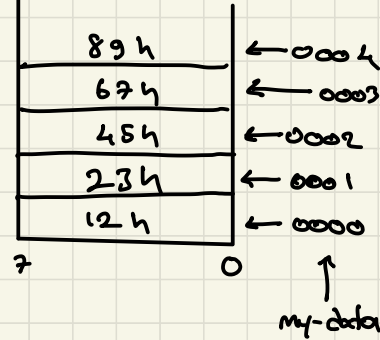
LDS:

LDS SI, my-data { [0000]

↓
da burada
- bilir

SI = 2312h ← registeri dolduruyor

DS = 6745h ← sonraki offset değerini
DS'ye atıyor



LDS:

LDS'nin aynısını yapıyor

DS yerine ES'ye atıyor

Data Segment - ekstra Segment

XCHG:

XCHG OP1, OP2

↳ Swap işlemi yapıyor OP1, OP2

MOV AX, 7
MOV BX, 9
XCHG AX, BX

XCHG RB, DADDR → XCHG AL, sayı
RW, DADDR → XCHG BX, sayı²
RW, RW → XCHG BX, AX → 2byte "4"
RB, RB → XCHG DH, CL
AX, RW → XCHG AX, BX → 1byte "8"

AX daha
uzak
çalışır!

ADD:

ADD OP1, OP2

$OP1 \leftarrow OP1 + OP2$

ÖR

MOV AX, 7
MOV BX, AX
ADD BX, 4x;

$BX = 14_{16}$

ADC:

ADD with Carry

ADC OP1, OP2

$OP1 \leftarrow OP1 + OP2 + CF$ → Carry flag

ÖR

MOV BX, 3; $CF \leftarrow 1$
ADC BX, 42

$BX \leftarrow 3 + 42 + 1 = 46_{16}$

ÖR

MOV AL, 0ffh → 1111111b → 255
MOV DH, 0Ch → +0001100b → 12
ADD AL, DH → 1001011b → 267

ADD AL, DH

CF Set

burası AL = 0001011b
AL = 11

ADC AH, 0

$AH \leftarrow 0 + 0 + 1 \rightarrow AH = 1$

00000001, 0001011 → 267
AH = 256 AL = 11

SUB:

SUB OP_1, OP_2

$OP_1 \leftarrow OP_1 - OP_2$

SBB:

SBB OP_1, OP_2

$OP_1 \leftarrow OP_1 - OP_2 - CF$

INC:

INC

RB

$\rightarrow$

$RB \leftarrow RB + 1$

RW

DADDR

* değeri 1 artırır!

ADD DH, 1

'de olabilir lakin

INC daha hızlı çalışır

INC

AX

$AX \leftarrow AX + 1$

Sayı

$Sayı \leftarrow Sayı + 1$

DEC:

INC 'in aynısı sadece 1 azaltır.

NEG:

NEG

DADDR

$OP \leftarrow 0 - OP$

RB

RW

Two's Complement.

CMP:

CMP OP_1, OP_2

$OP_1 - OP_2 \rightarrow$ Sonuç bayraklar da oluşur

MUL:

MUL OP
DADDR. S → source
RBS
RWS

destination → D

8/ MUL CL ← AX:AL x CL → 8 Bitlik
→ Çarpma kütlediğin diğer değeri MOV AL ye taşı

MUL SI ; DX, AX = AX x SI → 16 Bitlik
High Low

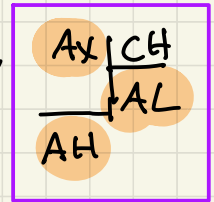
IMUL = işaretliyse

32 Bitlik işlemi sen
Çöz!

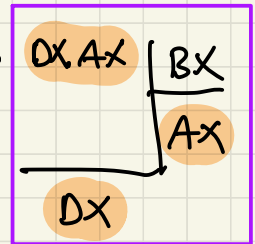
DIV:

DIV OP
DADDR
RBS
RWS

32 = DIV CH,



DIV BX,



DALLANMA KOMUTLARI

Koşulsuz Dallanma
komutları

* JMP

Koşullu Dallanma
komutları

Bir K.D.K

* JZ / JE
* JNZ / JNE
* JS
* JNS
* JO
* JNO
* JP / JPE
* JNP / JPO

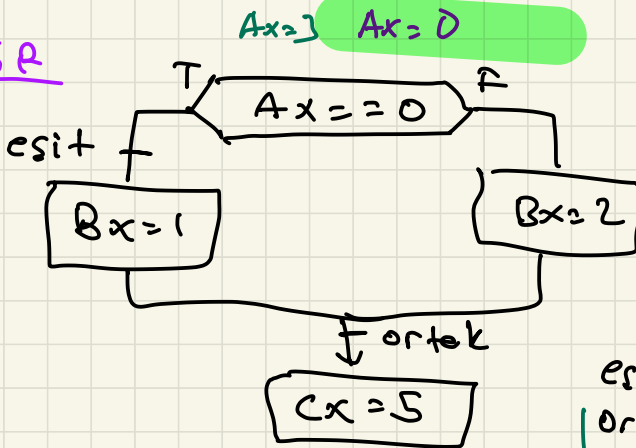
Birden fazla Bayrağın
Durumuna göre K.D.K.

İşaretsiz Sayılar
için Above/Below

İşaretili Sayılar
için Greater/Less

* JB / JNAE / JC * JL / JNGE
* JA / JNBE * JNL / JGE
* JAE / JNB / JNC * JLE / JNG
* JBE / JNA * JG / JNLE

ÖR

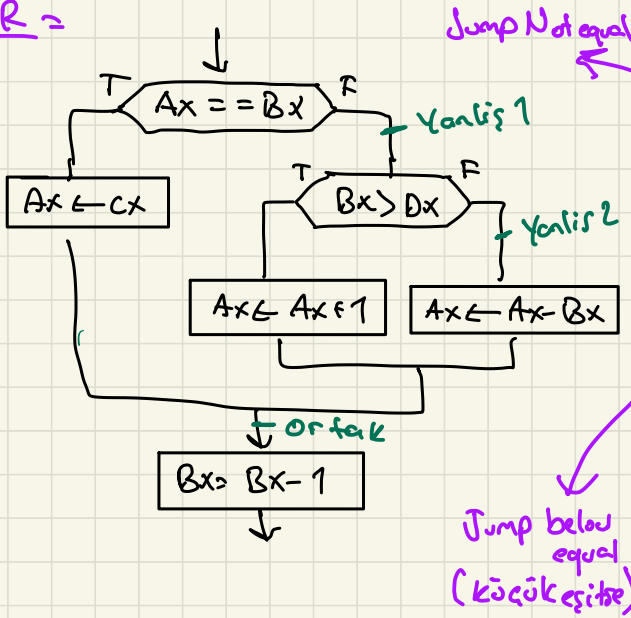


...

CMP Ax, 0
JE esit
MOV Bx, 2
JMP ortek

esit : MOV Bx, 1
ortek : MOV Cx, 5

ÖR =



CMP AX, BX

JNE yanlış 1

MOV AX, CX

JMP ortak

Yanlış 1: CMP BX, DX

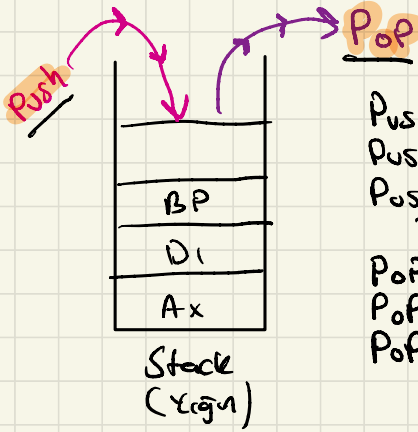
JBE yanlış 2

INC AX

JMP ortak

Yanlış 2: SUB AX, BX

ortak: DEC BX



Push AX

Push DI

Push BP

≡

Pop BX ← BP

Pop DI ← DI

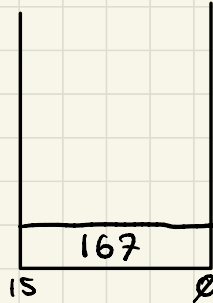
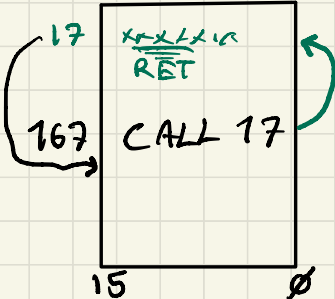
Pop SI ← AX

FIFO } First in Last Out

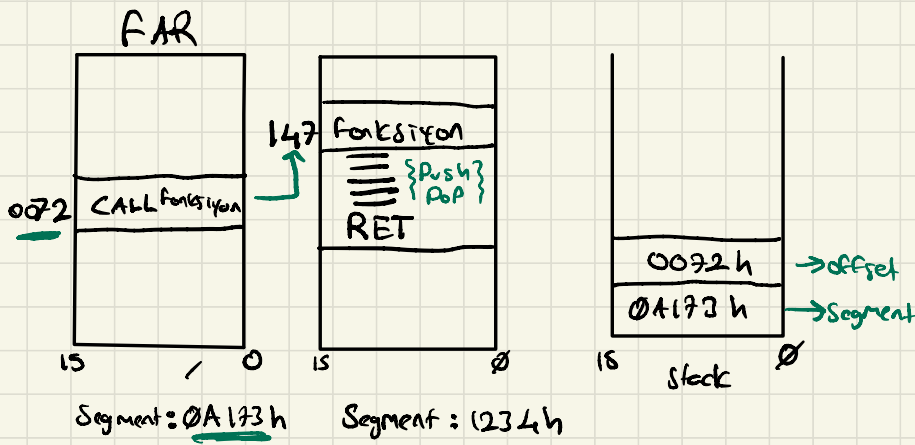
} işine alıp dışarı atıyor

CALL: { NEAR }
{ FAR }

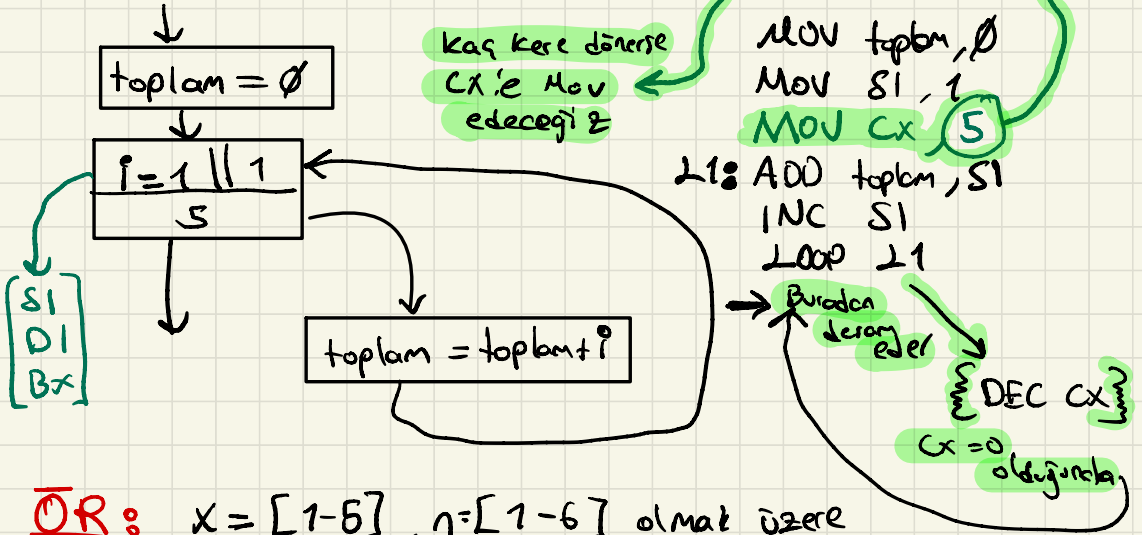
Near



Call komutu sağıldığı andaki offseti stack'e kayıtlar. ilgili adres gittikten sonra Return olur ve bir sonraki offset değerine gider "167"



LOOP:

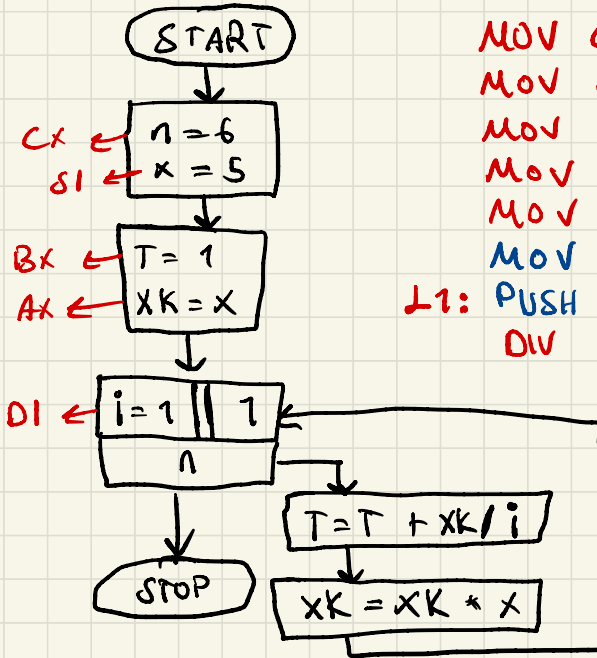


ÖR: $x = [1-5]$, $n = [1-6]$ olmak üzere

$$T = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \dots + \frac{x^n}{n}$$

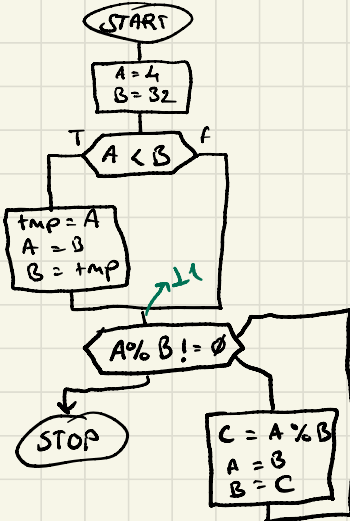
$T = ?$

Cevap:



MOV CX, 6
 MOV SI, 5
 MOV BX, 1
 MOV AX, SI
 MOV DI, 1
 MOV DX, 0 ← DIV için 0'la
 L1: PUSH AX → AX değerini saklamak için
 DIV DI
 ADD BX, AX → $BX = BX + AX$
 POP AX → XK
 MUL SI; DX:AX ← $AX * SI$
 INC DI $XK = XK * X$
 LOOP L1
 ; Sonuç BX'te

ÖR: A ve B [0-5000] arasında olmak üzere $EBOB(A, B)$



MOV AX, 4
 MOV BX, 32
 CMP AX, B
 JNB L1
 XCHG AX, BX
 L1: MOV DX, 0
 DIV BX
 CMP DX, 0
 JZ Son
 MOV AX, BX
 MOV BX, DX
 JMP L1
 Son; Sonuç

← Mantiksal Komputer: →

NOT OP

NOT A_x $A_x \leftarrow 00ffh$
 $A_x \leftarrow ff00h$

OR OP_1, OP_2

MOV AL, 12H
OR AL, 80H

12H → 0001 0010 b
80H → 1000 0000 b
OR
92H ← 1001 0010 b

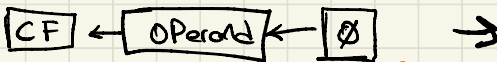
$$AL = gzh \text{ or } AL = 0h \text{ and}$$

AND OP_1, OP_2

XOR OP_1, OP_2

TEST $OP_1, OP_2 \rightarrow$ PSW $\in OP_1$ and OP_2
 some bayraklar

- SHL = Shift Left Logical:

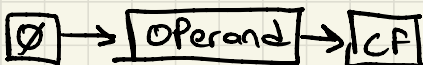


MOV AH, 7; 00000111
SHL AH, 1; 00001110

- 2 ile çarpmak

$$cf = \emptyset$$

- SHR: Shift Right Logical:



```
→ MOV CL, 2  
SHR BX, CL
```

- 2 ile bölme

OR = "Sayı" değerindeki değer çift ise $Ax = 0$
 " " " tek ise $Ax = 1$ yap.

SHR Sayı, 1
 JC Tek
 Mov $Ax, 0$
 JMP Son
 Tek: MOV $Ax, 1$
 Son:

Alternatif **TEST** Sayı, 1
 JNZ Tek

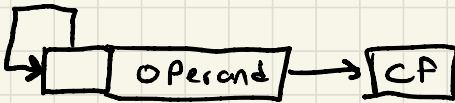
Sayı değeri konulu

Burada $Sayı \leftarrow Sayı / 2$ oldu

• SAL: Shift Arithmetic Left:

$SHL = SAL$

• SAR: Shift Arithmetic Right:



MOV AH, 10101100B
 SAR AH, 1

• negatif sayıları 2'ye böler

11010110 CF

• RCL: Rotate through Carry Left



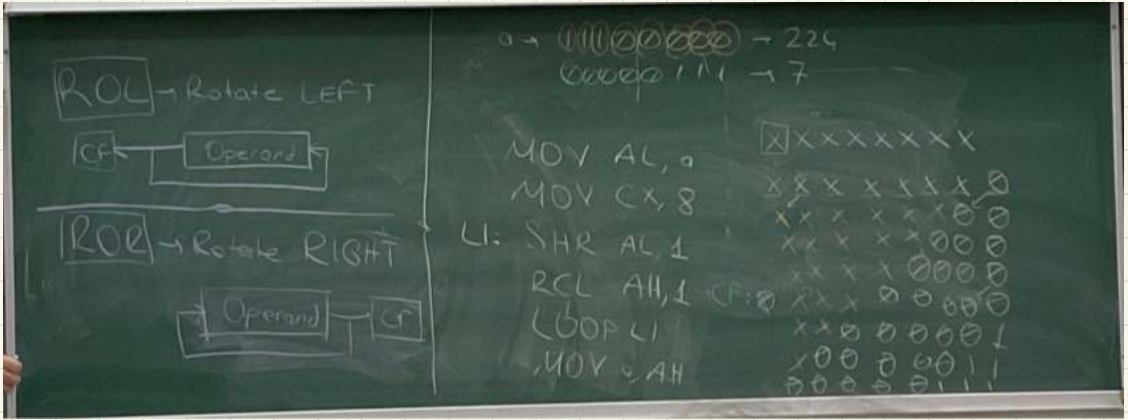
MOV AH, 10110101 b
 cdc
 RLC AH, 1

• RCR: Rotate through Carry Right



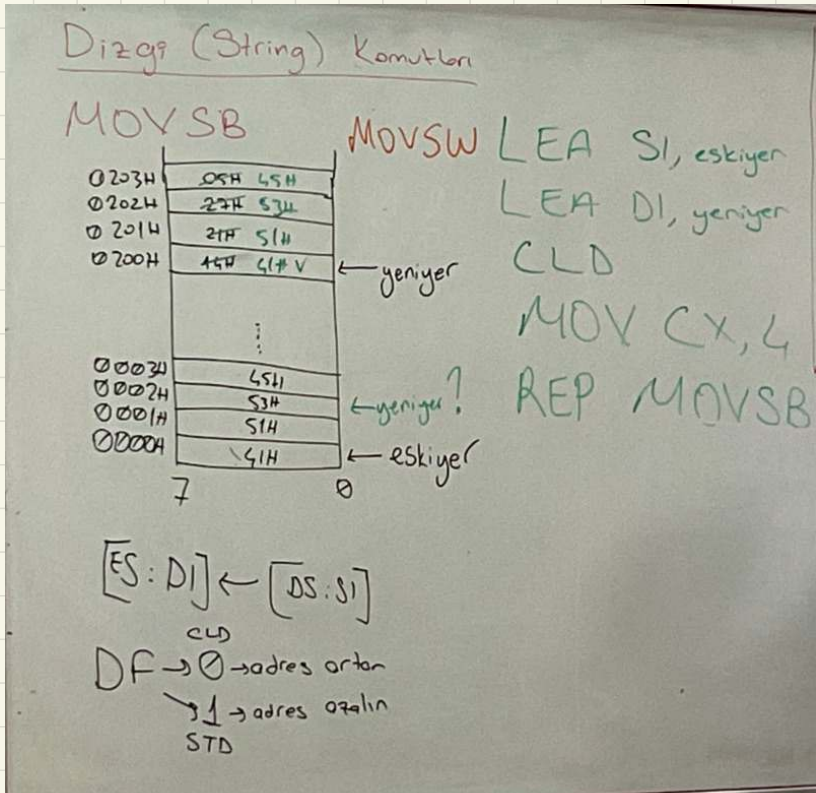
• ROL: Rotate Left

• ROR: Rotate Right



STRING İŞLEMLERİ

MOVSB



CMP SB:

CMP SB CMPSW

[DS:SI] - [ES:DI]

DF → 0 ↑
 1 ↓ n:10

LEA SI, dizil

LEA DI, dizir2

MOV CX, 10

CLD

REPE CMPSB

→ ; CX'e göre karşılaştırma

dizil	1	2	3	4	5	6	7	8	9	0
dizir2	1	2	3	4	5	6	7	8	9	0
	1	2	3	4	5	6	7	8	9	0

0 0 0 0 0 0 0 0 0 0 0

SCASB: (scan)

AL - [ES:DI] ⇒ Sonuç Bayraklarda oluşuyor

DF → 0 → adres artar
 1 → adres azalır

CH	50H
BH	6FH
AH	67H
9H	2EH
8H	43H
7H	46H
6H	40H
5H	61H
4H	6DH
3H	75H
0202H	79H
0201H	55H

7 ← 0 → e-mail

LEA DI, e-mail

Clear ← CLO

Direction Flag MOV CX, 12 → 12 kere tekrar

MOV AL, 'g'

esit olmadığı sürece tekrar et ← REPNE SCASB

→ CX değerini kontrol ederek aranan string değerini bulup bulmadığını anlayabiliriz

LODSB: (Load)

AL ← [DS:SI]

LEA SI, e-mail

LODSB ; AL ← SI

← SI'deki adres değerini AL'e atıyor

STOSB: (Store)

[ES:DI] ← AL

← AL'deki değeri Store ediyor

CBBW: (convert Byte to Word)

AX ← AL

← AL'nin değerini AX'e genişletiyor
AH değeri varsa yok ediyor

AL'nin MSB'si

1 ise → AH = 11111111b

0 ise → AH = 00000000b

CWD: (convert word to Double word)

$Dx: Ax \leftarrow Ax$

PUSHF: (Flaglerin içindeki değeri stack'e koyar)

POPF: (Flaglerin " " stackten alır)

LAHF: (Load AH with flag)

SF	ZF	?	AF	?	PF	?	CF
----	----	---	----	---	----	---	----

SAHF: (Store AH to Flag)

ÖR: Diğer bayraklara dokunmadan ZF değerinin complementini alalım

PUSHF

LAHF

XOR AH, 40H

POPF

SAHF

S Z X A X P X C

1 0 7 0 1 1 1

0 1 0 0 0 0 0

AH $\leftarrow$ 1 1 1 0 1 1 1

ÖR Verilen bir byte'teki bitleri sayan asm kodunu yazalım.

```
XOR BL,BL
MOV CX,8
MOV AH,55H
L1: SHL AH,1
    JNC devam
    INC BL
devam: LOOP L1
```

V_1

```
XOR BL,BL
MOV CX,8
MOV AH,55H
L1: SHL AH,1
    ADC BL,0
    Loop L1
```

V_2

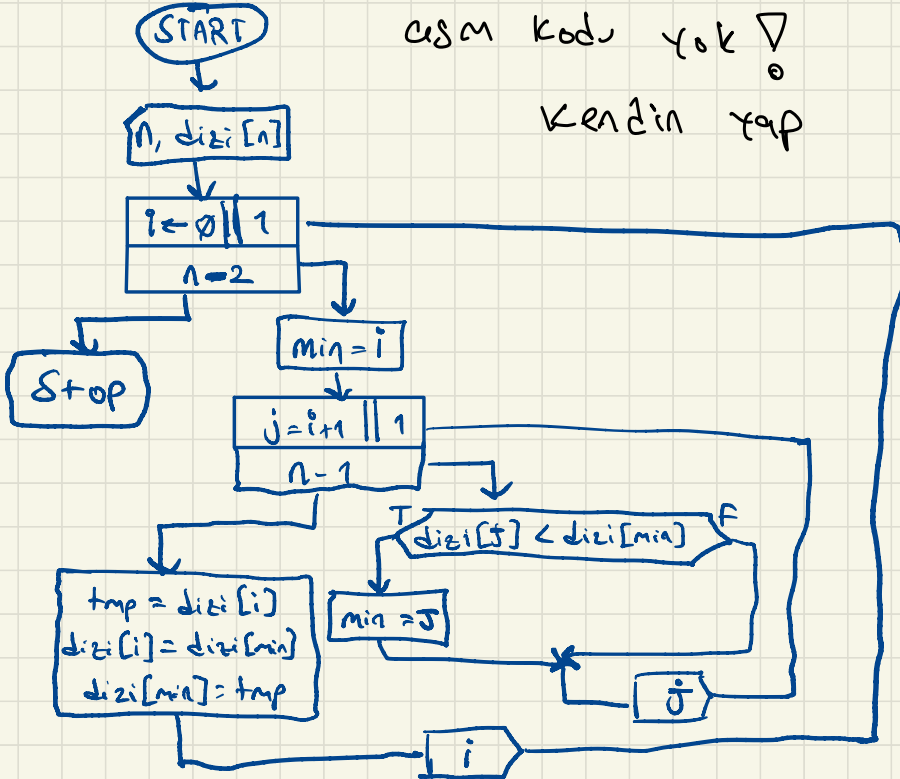
```
XOR BL,BL
MOV AH,55H
L1: CMP AH,0
    JE Cıkıs
    SHL AH,1
    ADC BL,0
    JMP L1
```

Cıkıs:

V_3

ÖR

asm kodu yok !
kendin yap



ADRESLEME MODLARI

Anlık (immediate) Adres

MOV {reg
daddr}, idata

MOV AX, 12

MOV yeni, -5

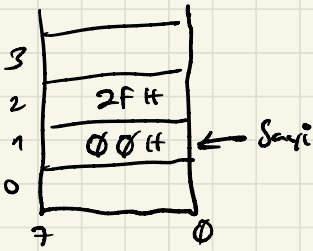
Yazmaç (Register) Adres

MOV reg, reg

MOV BP, SP

MOV AH, DL

Doğrudan (Direct) Adres



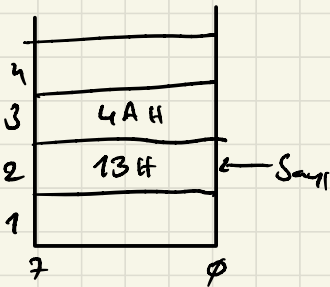
* Sayı WORD ise

MOV AX, Sayı ; $AX \leftarrow 2F00H$

* Sayı BYTE ise

MOV BL, Sayı ; $BL \leftarrow 00H$

Yazmaç Dolaylı (Register Indirect) Adres



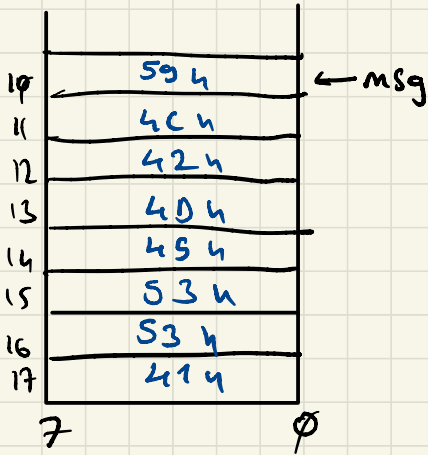
LEA SI, Sayı ; $SI: 0002$

MOV AX, [SI] ; $AX: 4A13H$

MOV DI, OFFSET Sayı ; $DI: 2$

MOV AX, [DI] ; $AX: 4A13H$

BA2 GÖRDELİ (BASE Relative) Adres



```
LEA BX, msg ; BX: 0x10h  
MOV AX, [BX+4]; AX: 5345h  
MOV CH, [BX+5]; CH: 53h  
MOV SI, 2  
MOV BP, [BX+SI+1]; BP: 4540h
```

Dogrudañ İndirli (Diret Index) Adres

dizi DB 41h, 53h, 54h, 45h

```
XOR SI, SI  
MOV AL, dizi[SI]; AL: 41h  
INC SI  
MOV AX, dizi[SI+1]; AX: 4554h
```

İskele (Port) Adres

IN / OUT

Diğzi (String) Adres

2 dizinin yerini farklı adr. tipleri ile değiştirelim.

d1, d2 → byte tanımlı

n → dizilerin boyutu

MOV CX, n

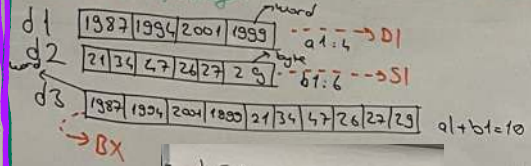
XOR SI, SI ; d1 adresi (Direct Index)

LEA DI, d2 ; d2 adresi (Register Indirect)

L1: MOV AL, d1[SI]
XCHG AL, [DI]
MOV d1[SI], AL
INC SI
INC DI
LOOP L1

MOV A

A şirketinde çalışanların doğum yılı d1 dizisinde, B şirketinde çalışanların yaşları d2 dizisinde tutulmaktadır. Önce A şirketinde çalışan bilgileri boyutu a1=b1 olan d3 dizisine aktarılacak.



LEA DI, d1
LEA SI, d2
XOR BX, BX
MOV CX, a1
L1: MOV AX, [DI]
MOV DS[BX], AX
ADD BX, 2
ADD DI, 2
LOOP L1
MOV CX, b1
L2: MOV AL, [SI]
CBW
MOV DS[DI], AX
ADD DI, 2
INC SI
LOOP L2

SÖZDE (PSEUDO) KOMUTLAR

• 1ST

Segment / Ends

Kesim - ismi

SEGMENT { seçenekler }

Kesim - ismi

ENDS

* Hizalama (align) tipi

BYTE 1
WORD 2
PARA 16
PAGE 256

* Birleştirme (Combine) tipi

→ PUBLIC
→ COMMON
→ STACK → LIFO
→ AT

* Sınıf (class) tipi

→ PUBLIC ise 'name'

stack sg SEGMENT PARA STACK 'yigin'

stack sg ENDS

code sg SEGMENT PARA 'kod'

code sg ENDS

data sg SEGMENT BYTE AT 012A0H

data sg ENDS

ASSUME

ASSUME SS : stack sg , DS : data sg , CS : code sg

DB (define Byte)

Sayi 1	DB	25
Sayi 2	DB	1E H
Sayi 3	DB	?
Dizi b 1	DB	1, 2, 3 F H
str b	DB	"Assembly ders!"

DW (define Word)

→ Sayılı DW Q ABCD h

→ Dizi w1 D w S, 12, 20

DD (define double word)

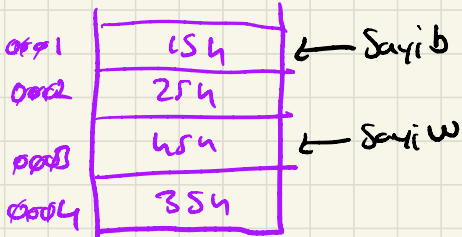
Dup (duplication factor)

→ diz: b2 DB IS DUP(0); diz: b2: 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

→ mb 1 DB 3 DP(4 DP(8));

→ Ditzu Ow w Dup(?) 12B ∈ Mbl : 8888
8888
8888

PTR (Pointer)



Sayib	DB	184
Sayim	DB	35454

MOV AX, WORD PTR SAYIB

MOV AX, BYTE PTR Seg: w

MOV AL, BYTE PTR [SI+1]

LABEL

DS {
Yeni S LABEL BYTE
Sayi w DW 2535h
Yeni w LABEL WORD
Sayi bb DB 45h
Sayi cc DB 55h

CS {
MOV AX, Yeni w
→ 5545h
MOV BL, Yeni b
→ 35h

PROC (Prosedür) / ENDP

yordam-ismi PROC {Near | FAR}
=====
RET / RETF
yordam-ismi ENDP

LENGTH

→ DUP: ten önceki değeri alır
→ Tablo DW IS DUP(?)

MOV AX, LENGTH Tablo
↓
000Fh

TYPE

sizeof(C)

Tablo DW IS DUP(?)
MOV AX, TYPE Tablo

↓
0002h
Sayi DD 3412ABCDh
MOV AX, TYPE Sayi
→ 0004h

SIZE

LENGTH * TYPE

Tablo DW 20 DUP(?)
MOV AX, SIZE Tablo

↓
 $20 * 2 = 40$

OFFSET

MOV AX, OFFSET (Tablo)

adres değerini AX'e atar

~~diğer [AX]~~ yapılmaz

SEG

Segment değeri
giriş

END

MOV ES, SEG Sayı

END baslangicxordame

MOV AX, SEG Aramafark

→ myds SEGMENT PARA 'veri'

orneksayi DB 17

ornekw2 DW 0ABCDH

≡

→ myds ENDS

→ myss SEGMENT PARA 'stack' 'yigin'

DW 20 DUP(?)

→ myss ENDS

→ mycs SEGMENT PARA 'kod'

ASSUME SS: myss, CS: mycs, DS: myds

ilk PROC PROC FAR ; int main() {

PUSH DS

XOR AX, AX

PUSH AX

MOV AX, myds

MOV DS, AX

DS: 0

1F4H

0000H

RET

ilk PROC ENDP

ikinci PROC NEAR

RET

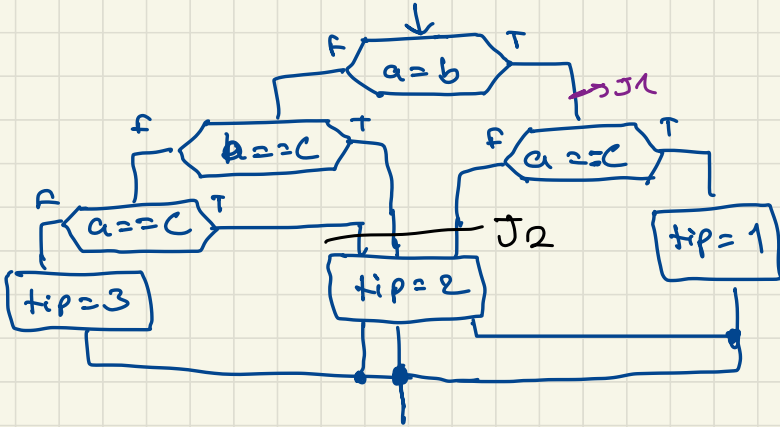
ikinci ENDP

mycs ENDS

END ilkPROC

ÖR

3 kenar boyutu bilinen bir değerin tipini bulalım.
"Eşkenar, ikizkenar, Genişkenar"



myds

SEGMENT PARA "veri"

a DB 17
b DB 12
c DB 12
tip DB (?)

myds

ENDS

myss

SEGMENT PARA STACK "yığın"

DB 12 DUP(?)

myss

ENDS

mycs

SEGMENT PARA 'kod'

ASSUME SS: myss, DS: myds, CS: mycs

LCGBN

PROC FAR

PUSH DS

XOR AX, AX

PUSH AX

MOV AX, myds

MOV DS, AX

MOV AL, a

MOV BL, b

MOV CL, c

```

CMP AL, BL
JE J1
CMP BL, CL
JE J2
CMP AL, CL
JE J2
MOV tip, 3
JMP Son
J1 CMP AL, CL
JNE J2
MOV tip, 1
JMP son
J2: MOV tip, 2
son: RETA

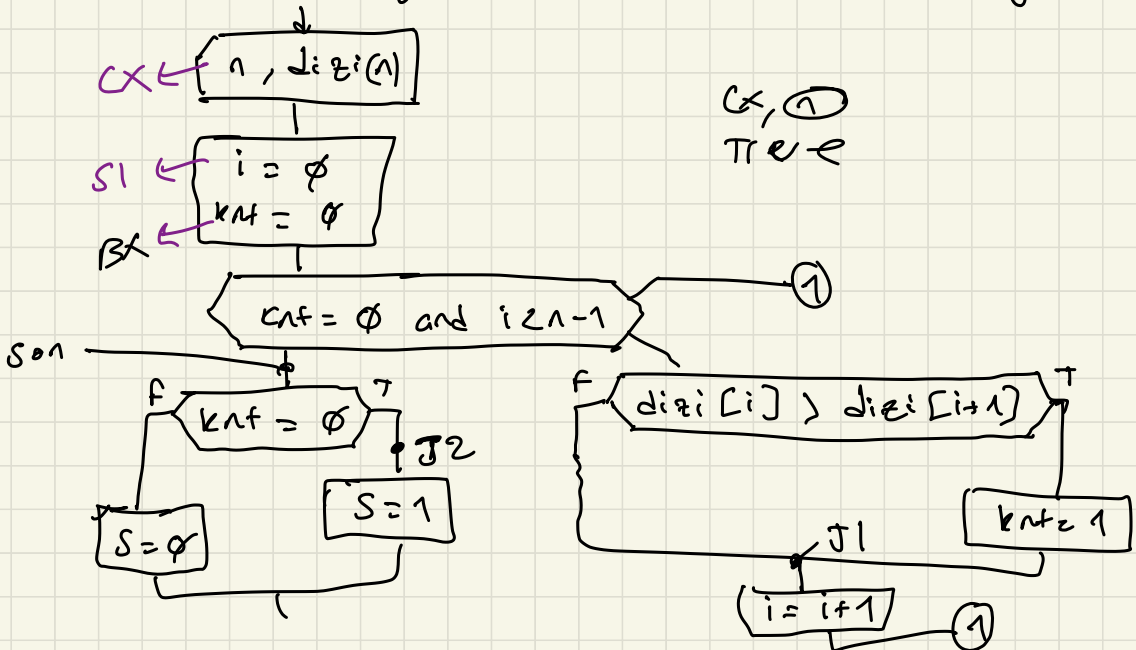
```

WEGEN
myCS

ENDP
ENDS
END UCGEN

BR

Byte tanımlı işaretli bir dizinin küçükten büyüğe sıralı olup olmadığını bulan exe tipi asm kodunu yazınız



my ds SEGMENT PARA 'veri'

 dizi DB 1,2,3,4,5

 n DB 5

 s DB (?)

my ds ENDS

my ss SEGMENT PARA STACK 'yigin'

 DB 12 DUP(?)

my ss ENDS

my cs SEGMENT PARA 'kod'

ASSUME SS: myss, CS: mycs, DS: myds

SIRA PROC FAR

 PUSH DS

 XOR AX, AX

 PUSH AX

 MOV AX, myds

 MOV DS, AX

 XOR SI, SI

 XOR BX, BX

 MOV CX, n

 DEC CX

etiket: CMP BX, 0

 JNE son

 CMP SI, CX

 JAE son

 MOV DL, dizi[SI]

 CMP DL, dizi[SI+1]

 JLE J1

 INC BX

J1: INC SI

 JMP etiket

son: CMP BX, 0

 JZ J2

 MOV S, 0

 JMP enson

J2: MOV S, 1

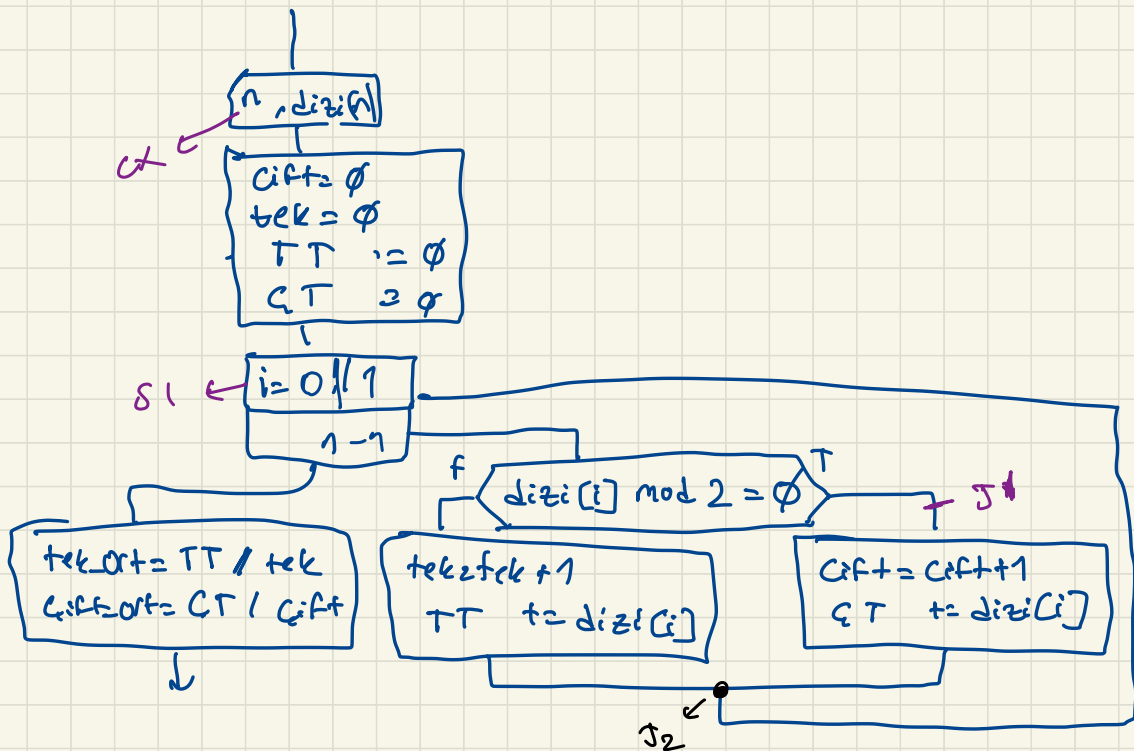
enson: RETF

SIRA ENDP

mycs ENDS

END SIRA

ÖR Bir dizideki word türüdeki teklerin ve çiftlerin ortalamasını bulan asm kodunu yazın.



```

myds SEGMENT PARA 'veri'
Dizi DW 2FFFh, 7AB2h, 70ABh, 7111h, 71fAh, 7232h,
       74F8h, 78CSh, 7S34h, 7d60h
çift DB 0
Tek DB 0
TT DB 0
CT DB 0
n DB 10
TOrt DB 0
çOrt DB 0

```

```

myds ENDS
myss SEGMENT PARA STACK 'yığın'
DB 20 DUP(?)
myss ENDS

```

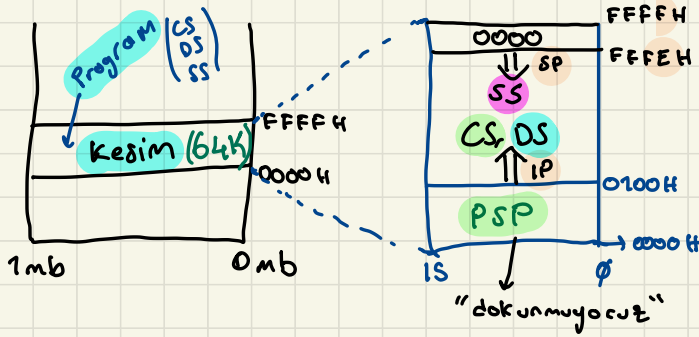
```

myCS SEGMENT PARA "cod"
ASSUME SS:mySS, CS:myCS, DS:myDS
ANA PROC FAR
    PUSH DS
    XOR AX, AX
    PUSH AX
    MOV AX, myDS
    MOV DS, AX
    MOV CX, n
    DEC CX
    XOR SI, SI
L1: MOV AX, SIZE [SI]
    TEST AX, 1
    JE J1
    INC Tek
    ADD WORD PTR [TT], AX
    ADC WORD PTR [TT+2], 0
    JMP SON
J1: INC Cift
    ADD WORD PTR [CT], AX
    ADC WORD PTR [CT+2], 0
SON: ADD SI, 2
    LOOP L1
    MOV DX, WORD PTR [TT+2]
    MOV AX, WORD PTR [TT]
    DIV Tek
    MOV Tort, AX
    MOV DX, WORD PTR [CT+2]
    MOV AX, WORD PTR [CT]
    DIV Cift
    MOV Cort, AX
    RETF
ENDP
ENDS
END ANA

```

ANA
myCS

Com tipi Program (compact)



Syntax - Model 1

```
codesg SEGMENT PARA 'kod'
      ORG 1000H
      ASSUME CS: codesg, SS: codesg, DS: codesg
```

Bilgi: JMP BASLA

```
Sayib1    DB ?
Sayiw1    DW ?
          :
```

```
BASLA    PROC NEAR
```

```
          :
```

```
          RET
```

```
BASLA
```

```
codesg
```

```
ENDP
```

```
ENDS
```

```
END Bilgi
```

Syntax - Model 2

```
mys SEGMENT PARA 'k'
    ORG 1000H
    ASSUME SS: mys, CS: mys, DS: mys
```

```
ANA PROC NEAR
```

```
    ==
```

```

    RET
ANA ENOP
Soyib1 DB ?
Soyiw2 DW ?
    ==
    ==
mys ENDS
END ANA

```

ÖR: 1 yıl boyunca günlük en düşük hava sıcaklığı bilgileri bulunan bir dizide -20'nin altında sıcaklık değeri olup olmadığını bulan asm programını COM tipinde yazalım

```

mys SEGMENT PARA 'k'
ORG 100h
ASSUME CS:mys,DS:mys,SS:mys
ANA
    RET
    ENOP
    DB ?
    DW ?
    ...
mys ENDS
END ANA

```

Soru 1 yıl süresince günlük en düşük hava sıcaklığı bilgileri bulunan bir dizide -20'nin altında sıcaklık değeri olup olmadığını bulan ASM programı COM tipinde yazalım

```

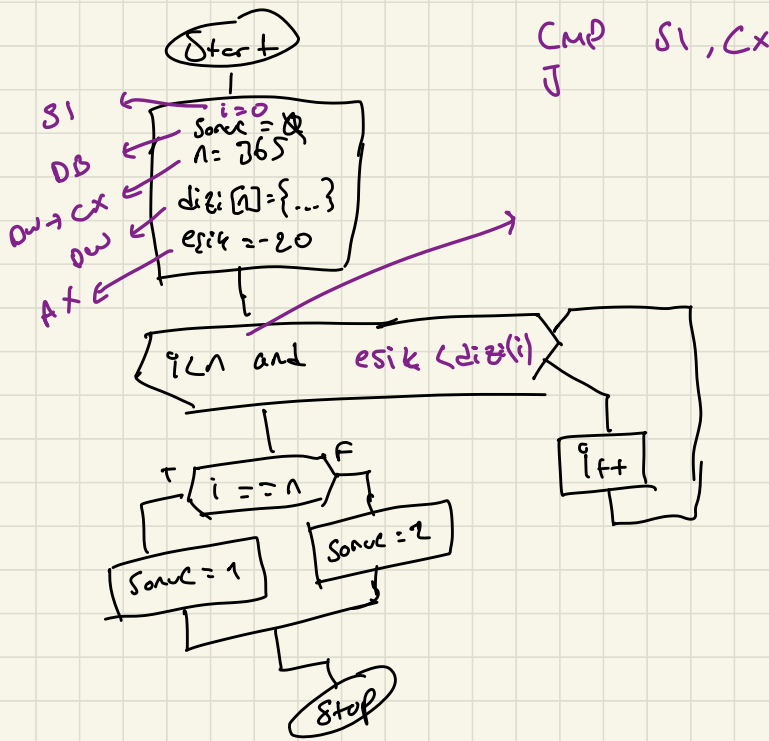
graph TD
    START([START]) --> Init[n = 365  
i = 0  
dizi[0] = -20]
    Init --> Loop{İçin 00 dizi[i] > esik}
    Loop -- Evet --> IncI[i = i + 1]
    Loop -- Hayır --> DecS[S = S - 1]
    DecS --> Loop
    IncI --> Loop
    DecS --> End([Dön])
    
```

CMP dizi[i], AL
CMP AL, dizi[0]

```

codesg SEGMENT PARA 'k'
ORG 100h
ASSUME CS:codesg,SS:codesg,DS:codesg
basla: JMP SICAKLIK
        DW 365
        DB 365 DUP(?)
        DB -20
        DB ?
SICAKLIK PROC NEAR
    XOR SI,SI
    MOV CX,n
    MOV AL,esik
don:     CMP SI,CX
        JAE yahis
        CMP AL,dizi[SI]
        JG  dogru
        INC SI
        JMP don
        yahis: MOV SI,2
        JMP don
        dogru: MOV SI,2
        RET
        SICAKLIK ENOP
        ENDS basla

```



CodeSeg SEGMENT PARA 'kod'
 ORG 1000H
 ASSUME SS: codeSeg, DS: codeSeg, CS: codeSeg
 Bilgi: JMP BASLA

Sonuc DB 0
 N DW 365
 esik DB -20
 dizi DB 365 DUP(?)

BASLA PROC NEAR

Sorgu: XOR SI, SI
 MOV CX, N
 MOV AL, esik
 CMP SI, CX
 JAE etiket1
 CMP AL, dizi[SI]
 JGE etiket2


```

INC SI
Jmp dogu
etiket1: mov sonuc, 1
        Jmp son
etiket2: mov sonuc, 2
son: RET
BASLA   ENDP
codeseg ENDS
        END   Bilgi

```

15_7 12
1 1 1 0 1000

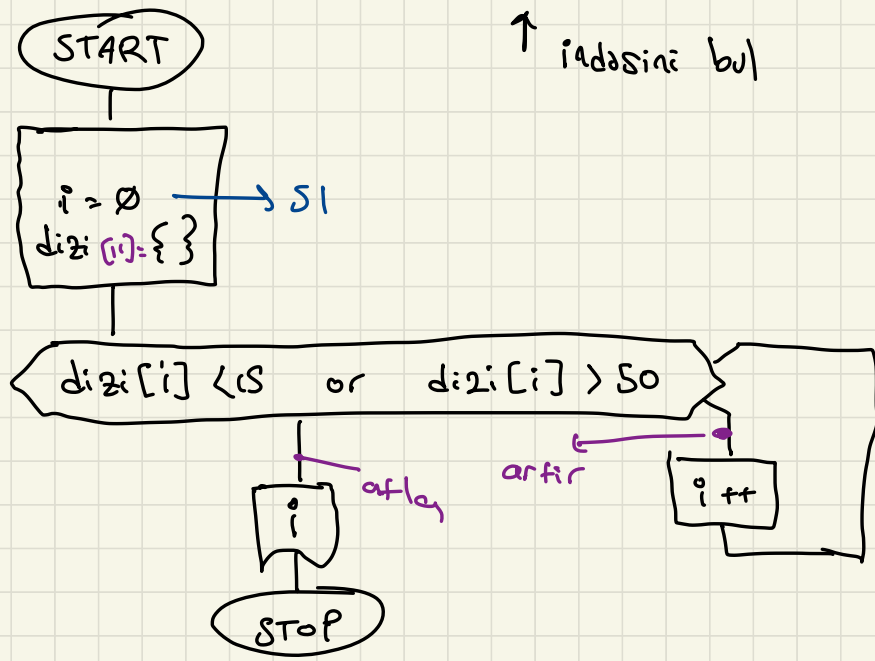
1110 1000
0001 0100
↓

16 + 4

20

ÖR 15 yaşından küçük ve 50 yaşından büyük 10 kişinin olduğu sıraya yaş koşulu bozan 11. kişi karışmıştır. Sıradaki tüm kişilerin yaşları bilindiğine göre sıraya sonradan karışan kişinin baştan kaçıncı sırada olduğunu bulan COM tipi ASM kodunu yazınız.

dizi [10] { 49, 14, 12, 8, 3, 52, 56, 70, 62, 63, }
 ↑ iadesini bul 64 }



```
mys SEGMENT PARA 'K'
ORG 100H
ASSUME CS:mys, DS:mys, SS:mys
BWL PROC NEAR
```

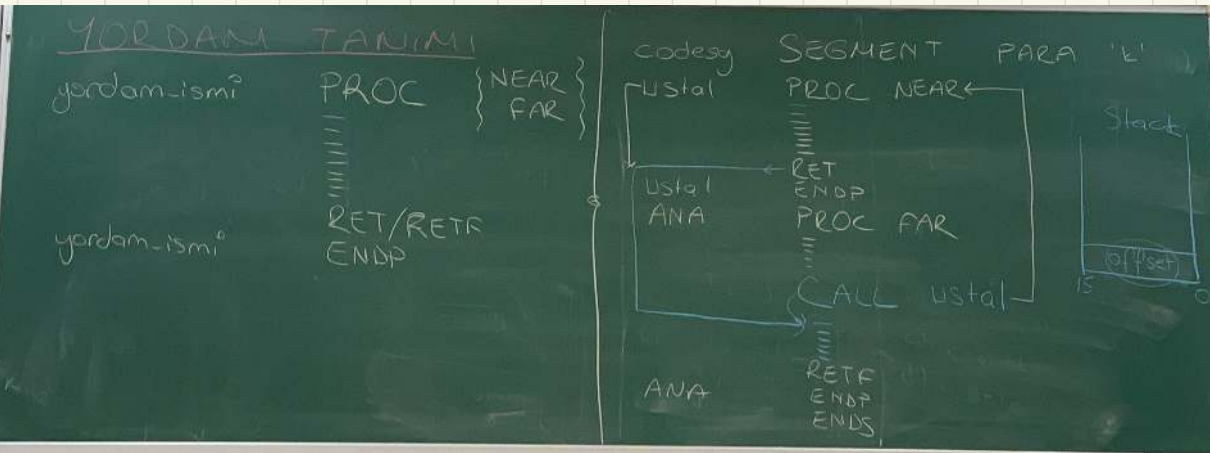
```
    XOR     SI, SI
don: MOV     AL, diz[i[SI]
      CMP    AL, 15
      JB     artik
      CMP    AL, 50
      JBE     atla
artik: INC SI
      JMP    don
atla: MOV    ind[i, SI
```

```
    RET
BWL ENDP
```

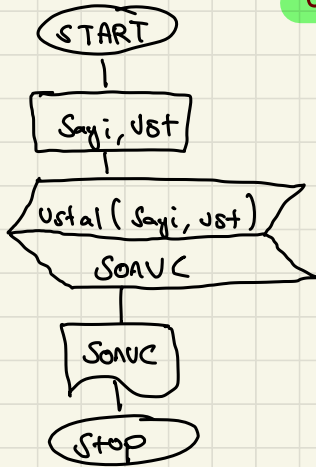
```
dizi DB 11, 10, 14, 8, 40, 3, 57, 62, 70, 12, 55
ind[i PW (?)
```

```
mys ENDS
END BWL
```

Yordam Tanımı



ÖR



üst alan bir asm kodu yazınız.

my ss

SEGMENT PARA STACK 'yigita'
 DB 20 DUP(?)

my ss
 my ds

ENDS
 SEGMENT PARA 'Veri'
 Sayi Dword 3
 us Dword 4
 Sonuc Dword (?)
 ENDS

my ds
 my cs

SEGMENT PARA 'kod'
 ASSUME SS:myss, DS:myds, CS:mycs

→ ANA

PROC FAR
 PUSH DS
 XOR SI, SI
 MOV AX, myds
 MOV DS, AX
 MOV BX, Sayi
 MOV CX, us
 CALL USTAL

→

ANA

USTAL

MOV Sonuc, AX

RETF

ENDP

PROC NEAR

MOV AX, 1

L1: MVL BX

LOOP L1

RET

→ USTAL ENDP

mycs ENDS

ENDANA

ÖR BH, BL üzerinden aldığı 2 değeri topluyor. FAR tipi
ASM yardımını ve ANA yardımını yazalım.

```
my ss    EXTRN TOPLAMA : FAR
          SEGMENT PARA STACK 'yığın'
          DB 20 DUP(?)
my ss    ENDS
my ds    SEGMENT PARA 'Veri'
          Sayi1 DB 3
          Sayi2 DB 4
          sonuc DW (?)
my ds    ENDS
my cs    SEGMENT PARA 'kod'
          ASSUME SS:myss, DS:myds, CS:mycs
          PROC FAR
          PUSH DS
          XOR AX, AX
          PUSH AX
          MOV AX, myds
          MOV DS, AX
          MOV BL, Sayi1
          MOV BH, Sayi2
          CALL TOPLAMA
          MOV sonuc, AX
          RET F
ANA      ENOP
my cs    ENDS
          END ANA
```

```
PUBLIC TOPLAMA
codes SEGMENT PARA 'kod'
        ASSUME CS:codes
        TOPLAMA PROC FAR
                XOR AX, AX
                MOV AL, BL
                ADD AL, BH
                ADC AH, 0
                RET F
        TOPLAMA ENDP
        codes ENDS
        END
```

DosBox'ta iki dosyayı da masm ile derliyoruz.

Link ederken → link ABC.obj + CDE.obj;

↓

sıralama öncelikli adımlarda exe dosyası oluşur

Macrolar

Makro-ismi

MACRO { parametreler }
LOCAL -----
ASSUME -----
≡ } Makro kodu
ENDM

TEST MAERO

XOR AX,AX
MOV DX,CX
≡

kod içinde her TEST gördüğünde oraya kapsalıyor

ÖR n elementli bir dizinin elemanlarını 2'ye bölen main yordamın içinde dizinin kaçak elemanları bulan Macro'yu COM tipi ASM olarak yazınız.

mycs **SEGMENT** PARA 'kod'
ORG 100H
ASSUME SS: mycs, DS: myds, CS: mycs

ANA PROC NEAR
XOR SI,SI
MOV CX,n
L1: SAR di[SI],1
INC SI
LOOP L1
ENCKK di:di,n
MOV kck,AL
RET
ENDP
ANA
di: DB 10,-2,4,6,10,3
n DW 6
kck DB (?)
mycs **ENDS**
END ANA

ENCKK **MACRO** di:di,n
LOCAL L1,don
XOR SI,SI
MOV AL,di:di[SI]
MOV CX,n
DEC CX
L1: CMP AL,di:di[SI]
JL don
MOV AL,di:di[SI]
don: INC SI
LOOP L1
ENDM